

Module 2 — Robustness Evaluation

Spectral-aware OOD robustness for the trained FNO (Burgers)
Team Turingz | Dharmapala R.D. | 2026-06-08

This document summarises two phases of work: (1) **building** Module 2 into the repository against its real interfaces, and (2) **running** the full pipeline on the real trained FNO checkpoint. It records what was built, what was corrected versus the starter script, the actual measured results, and the honest findings to carry into the viva.

1. What was built (3 files)

Rather than dropping in the starter script verbatim, the code was adapted to the repo's existing *single-source-of-truth* pattern under `common/` (alongside `canonical_split.py`, `metrics.py`, `reliability_signals.py`), so the PINN and DeepONet teammates reuse the same OOD spec and metric.

File	Role
<code>common/ood_spec.py</code>	The locked 3-input OOD spec — single source of truth, like <code>canonical_split.py</code> . PINN & DeepONet import the same cases + Cole–Hopf references.
<code>common/spectral_metrics.py</code>	The weighted Fourier-amplitude (spectral) distance $D(t)$ + early-warning crossing logic. The one new metric — the genuine contribution (no spectral metric existed).
<code>module2_robustness/robustness_eval.py</code>	The FNO runner. Loads the checkpoint, rolls out on each OOD IC, scores both metrics, writes CSV + JSON + 6-panel plot to <code>results/robustness/</code> .

2. What was corrected vs. the starter script

Reading the real code before writing caught several assumptions that would have produced wrong results:

- **Grid was wrong.** The starter hardcoded `np.linspace(0, 2, 200)`. The real t-grid is non-uniform ($[0, 0.01, \dots, 2.0]$) and x is $[-1, 1]$ with `endpoint=False` (periodic, 512 pts). The exact grid is now pulled from `colehopf.build_grid`, and the runner asserts the FNO's native grid matches before any pointwise comparison.
- **Cole–Hopf API.** The real call is `solve_burgers(x, t, u_ic, nu, L)`, not the imagined `generate_reference`. The reference loader is fully wired and verified — all three references generate finite, no ϕ_0 underflow, even at the shifted viscosity.
- **Reused shared rel-L2.** Uses `common.metrics._per_time_rel_l2` instead of re-implementing relative-L2, so FNO/PINN/DeepONet stay apples-to-apples.

3. The fixed OOD set

Case	Initial condition	Viscosity ν	Shift type
OOD-1	higher-frequency sinusoid	$1/(100\pi) \approx 3.18\text{e-}3$	IC family shift
OOD-2	localised Gaussian bump	$1/(100\pi) \approx 3.18\text{e-}3$	IC family shift
OOD-3	$\sin(\pi x)$, shifted viscosity	$1/(50\pi) \approx 6.37\text{e-}3$	parameter (ν) shift

Viva note: OOD-3's $\nu = 1/(50\pi)$ is $2\times$ the training viscosity $\rightarrow$ larger viscosity means a *smoother* solution, not sharper shocks. The starter's wording was loose. It is still a valid parameter shift — just describe its direction correctly.

4. Running it on the real checkpoint

The pipeline was executed end-to-end on the real trained checkpoint

`ml_models/fno/checkpoints/fno_burgers.pt`.

- **Environment:** neuralop lives in the conda env `fyp-pde` (`D:\miniforge3\envs\fyp-pde\python.exe`), not base python. Versions: neuralop 2.0.0, torch 2.7.1+cu118, numpy 2.4.4, scipy 1.17.1.
- **No version mismatch:** the earlier worry about a 2.2.1/1.26.4 pin did not materialise — the checkpoint loaded cleanly.
- **Grid guard passed:** $nx=512$, $nt=200$, $t \in [0, 2]$, train/extrapolation split at $t=1.0$.
- All three spectral weightings were run (`low_freq`, `uniform`, `high_freq`); outputs written to `results/robustness/` and per-weighting subfolders.

Commands used:

```
D:\miniforge3\envs\fyp-pde\python.exe module2_robustness\robustness_eval.py
... --weight-mode high_freq --out-dir results\robustness\high_freq
... --weight-mode uniform --out-dir results\robustness\uniform
```

5. Headline result — the FNO does not generalise out-of-family

Final-time ($t = 2$) errors. A relative-L2 above 1.0 means the prediction is wrong by more than the signal's own magnitude.

Case	IC	v	final rel-L2	final D (low_freq)
OOD-1	higher-freq sinusoid	$1/(100\pi)$	1.33	26.0
OOD-2	localised Gaussian bump	$1/(100\pi)$	0.97	15.1
OOD-3	$\sin(\pi x)$, shifted v	$1/(50\pi)$	0.49	32.1

This is the strong, defensible robustness finding: trained on one IC family, the operator collapses off it.

6. Honest finding — the early-warning lead does NOT come out positive

Across all three cases and all three weightings, the spectral early-warning lead is negative, undefined (nan), or zero — never the positive lead the guide hypothesised. The lead is defined as (time rel-L2 crosses its $2\times$ baseline) – (time D crosses its $2\times$ baseline); positive would mean D warns earlier.

Case	What happened	Lead
OOD-1	rel-L2 trips at $t=0.28$; D(low_freq) not until $t=1.12$. Under uniform/high_freq, D never crosses.	-0.83 (D lagged)
OOD-2	rel-L2 never crosses — its baseline is already 0.75 (FNO broken from $t=0$), so a $2\times$ -baseline rule cannot fire.	nan
OOD-3	The $t=0$ transient is the largest point in the baseline window, so both trip at $t=0$ trivially.	0

Root cause is structural, not a bug. The early-warning threshold assumes $[0, 0.25]$ is a trusted, in-distribution, low-error window to baseline against. But on these OOD inputs the FNO is already wrong inside that window (OOD-2 baseline rel-L2 = 0.75), so ' $2\times$ a broken baseline' is meaningless. Nudging the factor or baseline window will not fix it.

The real fix (a design change, not done unprompted): baseline each metric against an *in-distribution* run (the training IC family), not against the OOD run's own early window. Then D(t)'s departure can be compared fairly against rel-L2's.

7. What to take into the viva

- **Lead with the generalisation-failure numbers** (Section 5) — they are strong and clean.
- **Present the spectral lead honestly** as 'not supported under the self-baselined threshold; needs an in-distribution baseline' — do not claim a lead that the data does not show.
- **Describe OOD-3's ν correctly**: $2\times$ training viscosity $\rightarrow$ smoother, not sharper.
- **Do not hardcode a lead number** in the report before re-running with the in-distribution baseline fix.

Artifacts produced

- results/robustness/fno_robustness.csv, fno_robustness_timeseries.json, fno_robustness_plot.png (low_freq)
- results/robustness/high_freq/ and results/robustness/uniform/ (same three files each)

Generated for Team Turingz — Module 2 robustness evaluation. All numbers in this document are from the actual run on fno_burgers.pt in the fyp-pde environment.